
Section 1: Environment Setup

In [1]:
```
# Install packages if needed (uncomment if running locally)
!pip install transformers gensim scikit-learn numpy pandas
```

```
Requirement already satisfied: transformers in /Library/Frameworks/Python.
framework/Versions/3.12/lib/python3.12/site-packages (5.2.0)
Requirement already satisfied: gensim in /Library/Frameworks/Python.framew
ork/Versions/3.12/lib/python3.12/site-packages (4.4.0)
Requirement already satisfied: scikit-learn in /Library/Frameworks/Python.
framework/Versions/3.12/lib/python3.12/site-packages (1.8.0)
Requirement already satisfied: numpy in /Library/Frameworks/Python.framewo
rk/Versions/3.12/lib/python3.12/site-packages (2.4.2)
Requirement already satisfied: pandas in /Library/Frameworks/Python.framew
ork/Versions/3.12/lib/python3.12/site-packages (3.0.0)
Requirement already satisfied: huggingface-hub<2.0,>=1.3.0 in /Library/Fra
meworks/Python.framework/Versions/3.12/lib/python3.12/site-packages (from
transformers) (1.4.1)
Requirement already satisfied: packaging>=20.0 in /Library/Frameworks/Pyth
on.framework/Versions/3.12/lib/python3.12/site-packages (from transformer
s) (25.0)
Requirement already satisfied: pyyaml>=5.1 in /Library/Frameworks/Python.f
ramework/Versions/3.12/lib/python3.12/site-packages (from transformers)
(6.0.3)
Requirement already satisfied: regex!=2019.12.17 in /Library/Frameworks/Py
thon.framework/Versions/3.12/lib/python3.12/site-packages (from transforme
rs) (2024.9.11)
Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in /Library/Fra
meworks/Python.framework/Versions/3.12/lib/python3.12/site-packages (from
transformers) (0.22.2)
Requirement already satisfied: typer-slim in /Library/Frameworks/Python.fr
amework/Versions/3.12/lib/python3.12/site-packages (from transformers) (0.
24.0)
Requirement already satisfied: safetensors>=0.4.3 in /Library/Frameworks/P
ython.framework/Versions/3.12/lib/python3.12/site-packages (from transform
ers) (0.7.0)
Requirement already satisfied: tqdm>=4.27 in /Library/Frameworks/Python.fr
amework/Versions/3.12/lib/python3.12/site-packages (from transformers) (4.
67.2)
Requirement already satisfied: filelock in /Library/Frameworks/Python.fram
ework/Versions/3.12/lib/python3.12/site-packages (from huggingface-hub<2.
0,>=1.3.0->transformers) (3.20.3)
Requirement already satisfied: fsspec>=2023.5.0 in /Library/Frameworks/Pyt
hon.framework/Versions/3.12/lib/python3.12/site-packages (from huggingface
-hub<2.0,>=1.3.0->transformers) (2025.10.0)
Requirement already satisfied: hf-xet<2.0.0,>=1.2.0 in /Library/Framework
s/Python.framework/Versions/3.12/lib/python3.12/site-packages (from huggin
gface-hub<2.0,>=1.3.0->transformers) (1.2.0)
Requirement already satisfied: httpx<1,>=0.23.0 in /Library/Frameworks/Pyt
hon.framework/Versions/3.12/lib/python3.12/site-packages (from huggingface
-hub<2.0,>=1.3.0->transformers) (0.28.1)
Requirement already satisfied: shellingham in /Library/Frameworks/Python.f
ramework/Versions/3.12/lib/python3.12/site-packages (from huggingface-hub<
2.0,>=1.3.0->transformers) (1.5.4)
Requirement already satisfied: typing-extensions>=4.1.0 in /Library/Framew
orks/Python.framework/Versions/3.12/lib/python3.12/site-packages (from hug
gingface-hub<2.0,>=1.3.0->transformers) (4.15.0)
Requirement already satisfied: anyio in /Library/Frameworks/Python.framewo
rk/Versions/3.12/lib/python3.12/site-packages (from httpx<1,>=0.23.0->hugg
ingface-hub<2.0,>=1.3.0->transformers) (4.12.1)
Requirement already satisfied: certifi in /Library/Frameworks/Python.frame
work/Versions/3.12/lib/python3.12/site-packages (from httpx<1,>=0.23.0->hu
ggingface-hub<2.0,>=1.3.0->transformers) (2026.1.4)
Requirement already satisfied: httpcore==1.* in /Library/Frameworks/Pytho
n.framework/Versions/3.12/lib/python3.12/site-packages (from httpx<1,>=0.2
```

```
3.0->huggingface-hub<2.0,>=1.3.0->transformers) (1.0.9)
Requirement already satisfied: idna in /Library/Frameworks/Python.framewor
k/Versions/3.12/lib/python3.12/site-packages (from httpx<1,>=0.23.0->huggi
ngface-hub<2.0,>=1.3.0->transformers) (3.11)
Requirement already satisfied: h11>=0.16 in /Library/Frameworks/Python.fra
mework/Versions/3.12/lib/python3.12/site-packages (from httpcore==1.*->htt
px<1,>=0.23.0->huggingface-hub<2.0,>=1.3.0->transformers) (0.16.0)
Requirement already satisfied: scipy>=1.7.0 in /Library/Frameworks/Python.
framework/Versions/3.12/lib/python3.12/site-packages (from gensim) (1.17.
0)
Requirement already satisfied: smart_open>=1.8.1 in /Library/Frameworks/Py
thon.framework/Versions/3.12/lib/python3.12/site-packages (from gensim)
(7.5.1)
Requirement already satisfied: joblib>=1.3.0 in /Library/Frameworks/Pytho
n.framework/Versions/3.12/lib/python3.12/site-packages (from scikit-learn)
(1.5.3)
Requirement already satisfied: threadpoolctl>=3.2.0 in /Library/Framework
s/Python.framework/Versions/3.12/lib/python3.12/site-packages (from scikit
-learn) (3.6.0)
Requirement already satisfied: python-dateutil>=2.8.2 in /Library/Framewor
ks/Python.framework/Versions/3.12/lib/python3.12/site-packages (from panda
s) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /Library/Frameworks/Python.fram
ework/Versions/3.12/lib/python3.12/site-packages (from python-dateutil>=2.
8.2->pandas) (1.17.0)
Requirement already satisfied: wrapt in /Library/Frameworks/Python.framewo
rk/Versions/3.12/lib/python3.12/site-packages (from smart_open>=1.8.1->gen
sim) (2.1.1)
Requirement already satisfied: typer>=0.24.0 in /Library/Frameworks/Pytho
n.framework/Versions/3.12/lib/python3.12/site-packages (from typer-slim->t
ransformers) (0.24.0)
Requirement already satisfied: click>=8.2.1 in /Library/Frameworks/Python.
framework/Versions/3.12/lib/python3.12/site-packages (from typer>=0.24.0->
typer-slim->transformers) (8.3.1)
Requirement already satisfied: rich>=12.3.0 in /Library/Frameworks/Python.
framework/Versions/3.12/lib/python3.12/site-packages (from typer>=0.24.0->
typer-slim->transformers) (14.3.2)
Requirement already satisfied: annotated-doc>=0.0.2 in /Library/Framework
s/Python.framework/Versions/3.12/lib/python3.12/site-packages (from typer>
=0.24.0->typer-slim->transformers) (0.0.4)
Requirement already satisfied: markdown-it-py>=2.2.0 in /Library/Framework
s/Python.framework/Versions/3.12/lib/python3.12/site-packages (from rich>=
12.3.0->typer>=0.24.0->typer-slim->transformers) (4.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /Library/Framewo
rks/Python.framework/Versions/3.12/lib/python3.12/site-packages (from rich
>=12.3.0->typer>=0.24.0->typer-slim->transformers) (2.19.2)
Requirement already satisfied: mdurl~=0.1 in /Library/Frameworks/Python.fr
amework/Versions/3.12/lib/python3.12/site-packages (from markdown-it-py>=
2.2.0->rich>=12.3.0->typer>=0.24.0->typer-slim->transformers) (0.1.2)

[notice] A new release of pip is available: 26.0 -> 26.0.1
[notice] To update, run: pip install --upgrade pip
```

In [2]:
```
from transformers import pipeline, AutoTokenizer, AutoModelForSeq2SeqLM
import gensim.downloader as api
import numpy as np
import pandas as pd
import os
from sklearn.metrics.pairwise import cosine_similarity
```

```
from IPython.display import display, Markdown

from warnings import filterwarnings
filterwarnings('ignore')

import matplotlib.pyplot as plt
import matplotlib.colors as mcolors
import seaborn as sns

from evaluate import load
```

```
/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/site-pac
kages/tqdm/auto.py:21: TqdmWarning: IProgress not found. Please update jup
yter and ipywidgets. See https://ipywidgets.readthedocs.io/en/stable/user_
install.html
  from .autonotebook import tqdm as notebook_tqdm
```

In [3]:
```
print("Libraries imported successfully.")
```

```
Libraries imported successfully.
```

Section 2: Load Text Generation Model

In [4]:
```
generator = pipeline("text-generation", model="distilgpt2")
```

```
Loading weights: 100%|██████████| 76/76 [00:00<00:00, 1793.85it/s, Materia
lizing param=transformer.wte.weight]
```

Section 3: Text Generation Example

In [5]:
```
prompt = "AI is transforming industries by"
```

In [6]:
```
outputs = generator(
    prompt,
    max_length=50,
    num_return_sequences=3,
    do_sample=True,
    temperature=0.9,
    top_p=0.95
)
```

```
Passing `generation_config` together with generation-related arguments=
({'do_sample', 'temperature', 'max_length', 'top_p', 'num_return_sequence
s'}) is deprecated and will be removed in future versions. Please pass eit
her a `generation_config` object OR all generation parameters explicitly,
but not both.
Setting `pad_token_id` to `eos_token_id`:50256 for open-end generation.
Both `max_new_tokens` (=256) and `max_length`(=50) seem to have been set.
`max_new_tokens` will take precedence. Please refer to the documentation f
or more information. (https://huggingface.co/docs/transformers/main/en/mai
n_classes/text_generation)
```

In [7]:
```
print(f"Text Generation Outputs for {prompt}")
for i, output in enumerate(outputs):
```

```
    print(f"\nOutput {i+1}:")
    print(output['generated_text'])
    print("-"*50)
```

```
Text Generation Outputs for AI is transforming industries by

Output 1:
AI is transforming industries by transforming the environment. The new too
ls are a new way to communicate with your customers.
```

```
--------------------------------------------------

Output 2:
AI is transforming industries by introducing more flexible and more flexib
le labour structure.
```

```
------------------------------------------------

Output 3:
AI is transforming industries by eliminating the bureaucratic bureaucracy
to make them the top two percent of the workforce.
```

```
------------------------------------------------
```

Section 4: Tokenization Example

In [8]:
```
tokenizer = AutoTokenizer.from_pretrained("distilgpt2")
sentence = "LLMs are powerful tools for natural language understanding."
encoding = tokenizer(sentence)
```

In [9]:
```
token_ids = encoding["input_ids"]
tokens = tokenizer.convert_ids_to_tokens(token_ids)
seq_length = len(token_ids)
```

In [10]:
```
print("\nTokenization Example")
print("Sentence:", sentence)
print("\nTokens:", tokens)
print("\nToken IDs:", token_ids)
print("\nSequence Length:", seq_length)
```

```
Tokenization Example
Sentence: LLMs are powerful tools for natural language understanding.

Tokens: ['LL', 'Ms', 'Ġare', 'Ġpowerful', 'Ġtools', 'Ġfor', 'Ġnatural', 'Ġ
language', 'Ġunderstanding', '.']

Token IDs: [3069, 10128, 389, 3665, 4899, 329, 3288, 3303, 4547, 13]

Sequence Length: 10
```

Section 5: Prompt Engineering Examples

SUMMARISATION: USING GPT MODEL

In [11]:
```
# Summarization Prompt
prompt_summary = '''
Summarize the following text in 30 words or less:

Text:
True ambition is not merely the pursuit of accolades,
but a methodical dedication to refining one's craft.
Many individuals falter because they equate success with instantaneous re
overlooking the incremental progress necessary for mastery.
A truly ambitious person welcomes failure as a formative experience,
extracting lessons from every setback rather than succumbing to despair.
This tenacious mindset requires a delicate balance of confidence and humi
allowing for continuous adaptation in a rapidly changing environment.
Furthermore, prioritizing long-term goals over immediate gratification di
the visionary from the dreamer.
Ultimately, consistent effort, when paired with strategic thinking, yield
achievement, transforming raw talent into exceptional skill.

Summary:
'''
```

In [12]:
```
summary_output = generator(prompt_summary,
                           max_new_tokens = 60,
                           do_sample=True,
                           temperature=0.7,
                           top_p=0.9,
                           repetition_penalty=1.2
                           )
print("\nSummarization Output:")
processed_text = summary_output[0]["generated_text"]
summary = processed_text.replace(prompt_summary, "").strip()

print(summary)
```

```
Passing `generation_config` together with generation-related arguments=
({'do_sample', 'temperature', 'max_new_tokens', 'repetition_penalty', 'top
_p'}) is deprecated and will be removed in future versions. Please pass ei
ther a `generation_config` object OR all generation parameters explicitly,
but not both.
Setting `pad_token_id` to `eos_token_id`:50256 for open-end generation.
Both `max_new_tokens` (=60) and `max_length`(=50) seem to have been set. `
max_new_tokens` will take precedence. Please refer to the documentation fo
r more information. (https://huggingface.co/docs/transformers/main/en/main
_classes/text_generation)
Summarization Output:
It seems that this approach has been adopted by many people around our cou
ntry since childhood through education at Harvard University (IUC), and so
it may be worth reading more about how we are developing an effective stra
tegy based on these principles.
To learn further please visit www/crisisfortunes
```

SUMMARISATION USING SUMMARISTION MODEL (DistilBART)

In [13]:
```
def summarise_with_model(paragraph, model_name):
    """
    Summarise the input paragraph using the specified pre-trained model.
```

```
    - model_name: the name of the pre-trained model to use for summarizat
    - the function can be modified to accept additional parameters for ge
    - tokenizer initiated for model_name to allow for flexibility in whic
    """
    ## Load the model and tokenizer, and prepare the input
    tokenizer = AutoTokenizer.from_pretrained(model_name)
    model = AutoModelForSeq2SeqLM.from_pretrained(model_name)

    ## Tokenise the input paragraph
    inputs = tokenizer(paragraph, return_tensors='pt', truncation=True, m

    ## Generate the summary
    summary_ids = model.generate(inputs['input_ids'],
                                 max_length=60,
                                 min_length=20,
                                 length_penalty=2.0,
                                 num_beams=4,
                                 early_stopping=True
                                 )
    ## Decode the summary
    summary = tokenizer.decode(summary_ids[0], skip_special_tokens=True)

    return summary
```

In [14]:
```
models = ['sshleifer/distilbart-cnn-12-6', 'facebook/bart-large-cnn']

## loop through models, generate summaries, and store results in a DataFr
rows = []

## Append result from GPT2 generator for summary
rows.append({
    "model":"text generation distilgpt2",
    "summary":summary
})

## For each model, generate summary results
for model_name in models:
    summary = summarise_with_model(prompt_summary, model_name)
    rows.append({
        "model": model_name,
        "summary": summary
    })
df = pd.DataFrame(rows)
display(df)
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please se
t a HF_TOKEN to enable higher rate limits and faster downloads.
Please make sure the generation config includes `forced_bos_token_id=0`.
Loading weights: 100%|██████████| 358/358 [00:00<00:00, 2436.02it/s, Mater
ializing param=model.shared.weight]
The tied weights mapping and config for this model specifies to tie model.
shared.weight to model.decoder.embed_tokens.weight, but both are present i
n the checkpoints, so we will NOT tie them. You should update the config w
ith `tie_word_embeddings=False` to silence this warning
The tied weights mapping and config for this model specifies to tie model.
shared.weight to model.encoder.embed_tokens.weight, but both are present i
n the checkpoints, so we will NOT tie them. You should update the config w
ith `tie_word_embeddings=False` to silence this warning
Loading weights: 100%|██████████| 511/511 [00:00<00:00, 1577.84it/s, Mater
ializing param=model.encoder.layers.11.self_attn_layer_norm.weight]
```

	model	summary
0	text generation distilgpt2	It seems that this approach has been adopted b...
1	sshleifer/distilbart-cnn-12-6	A truly ambitious person welcomes failure as ...
2	facebook/bart-large-cnn	A truly ambitious person welcomes failure as a...

In [15]:
```
for _, row in df.iterrows():
    print(f'Model name: {row['model']}')
    print(row['summary'])
    print('-'*50)
```

```
Model name: text generation distilgpt2
It seems that this approach has been adopted by many people around our cou
ntry since childhood through education at Harvard University (IUC), and so
it may be worth reading more about how we are developing an effective stra
tegy based on these principles.
To learn further please visit www/crisisfortunes
--------------------------------------------------
Model name: sshleifer/distilbart-cnn-12-6
 A truly ambitious person welcomes failure as a formative experience . pri
oritizing long-term goals over immediate gratification distinguishes the v
isionary from the dreamer . Ultimately, consistent effort, when paired wit
h strategic thinking, yields lasting achievement .
--------------------------------------------------
Model name: facebook/bart-large-cnn
A truly ambitious person welcomes failure as a formative experience, extra
cting lessons from every setback rather than succumbing to despair. priori
tizing long-term goals over immediate gratification distinguishes the visi
onary from the dreamer. Consistent effort, when paired with strategic thin
king, yields lasting success.
--------------------------------------------------
```

In [16]:
```
## QUANTITATIVE EVALUATION OF SUMMARISATION

rouge = load("rouge")

summary_distilgpt2 = df['summary'].iloc[0]
summary_distilbart = df['summary'].iloc[1]
summary_bart = df['summary'].iloc[2]

predictions = [summary_distilgpt2, summary_distilbart, summary_bart]

reference_answer = ["A truly ambitious person welcomes failure as a forma
results = rouge.compute(
    predictions = predictions,
    references=[reference_answer]*3
)

print(results)
```

```
{'rouge1': np.float64(0.6103896103896104), 'rouge2': np.float64(0.59111111
11111111), 'rougeL': np.float64(0.6103896103896104), 'rougeLsum': np.float
64(0.6103896103896104)}
```

COMMENTARY ON DIFFERENT WAYS OF SUMMARISATION

- GPT2 prompt (DistilGPT2) can be used to demo ability to summarise

    - This is a general purpose model that predicts the next token in sequence
    - The objective of the model is next work prediction and langauge generation, not compression of text
    - The result **can** resemble summary, but often the result is hallucination (as seen above)
- Using model specifically trained to summarise like distilbart / large bart improves the results
    - These models are based on encode - decoder architecture
    - The models are fine tuned on data sets with document/summary pair (labelled data)
    - This training / fine tuning improves the quality of summarisation
- Task specific models should be adopted for best results

Model	Behaviour
DistilGPT2	Hallucinates
DistilBART	Concise
BART-large	Coherent

Q&A PROMPTS

USING GPT2

In [17]:
```
# Q&A Prompt
prompt_qa = '''
Question: What is the capital of France?
Answer:
'''

qa_output = generator(prompt_qa,
                      max_new_tokens=15,
                      temperature = 0.1,
                      repetition_penalty=1.3
                      )

processed_text = qa_output[0]["generated_text"]
answer = processed_text.replace(prompt_qa, "").strip()
print(answer)
```

```
Passing `generation_config` together with generation-related arguments=
({'repetition_penalty', 'max_new_tokens', 'temperature'}) is deprecated an
d will be removed in future versions. Please pass either a `generation_con
fig` object OR all generation parameters explicitly, but not both.
Setting `pad_token_id` to `eos_token_id`:50256 for open-end generation.
Both `max_new_tokens` (=15) and `max_length`(=50) seem to have been set. `
max_new_tokens` will take precedence. Please refer to the documentation fo
r more information. (https://huggingface.co/docs/transformers/main/en/main
_classes/text_generation)
France has a population that consists mainly in French citizens. The avera
ge number for
```

USING QA MODELS

In [18]:
```
qa_roberta = pipeline("question-answering", model="deepset/roberta-base-s
```

```
Loading weights: 100%|██████████| 199/199 [00:00<00:00, 2047.10it/s, Mater
ializing param=roberta.encoder.layer.11.output.dense.weight]
RobertaForQuestionAnswering LOAD REPORT from: deepset/roberta-base-squad2
Key                             | Status     |  |
--------------------------------+------------+--+-
roberta.embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architect
ure; not ok if you expect identical arch.
```

In [19]:
```
context = """
France is a country in Western Europe.
The capital city of France is Paris.
It is one of the most visited cities in the world.
"""

def generate_answers(question, context, qa):
    """
    Generate an answer to the question based on the provided context usin
    - question: the question to be answered
    - context: the context from which the answer should be derived
    - qa: the question-answering pipeline to use for generating the answe
    """
    result = qa(question=question, context=context)
    return {
        "answer": result["answer"],
        "confidence": result["score"]
    }
```

In [20]:
```
## Collect answers from all methods
qa_data = []
qa_data.append({
    "model":"distilgpt2",
    "question":"what is capital of France?",
    "answer":answer,
    "confidence":"unknown"
})

## Will fetch the correct answer
question = 'What is the capital of France?'
ans =  generate_answers(question, context, qa_roberta)
qa_data.append({
    "model":"roBERTa",
    "question":question,
    "answer":ans['answer'],
    "confidence":ans['confidence']
})

## Will not fetch the correct answer
question = 'What is the capital of England?'
ans =  generate_answers(question, context, qa_roberta)
qa_data.append({
    "model":"roBERTa",
    "question":question,
    "answer":ans['answer'],
```

```
    "confidence":ans['confidence']
})
```

In [21]:
```
df = pd.DataFrame(qa_data)
for _, row in df.iterrows():
    print(f'Model: {row['model']}')
    print(f'Question: {row['question']}')
    print(f'Answer: {row['answer']}')
    print(f'Confidence: {row['confidence']}')
    print('-'*50)
```

```
Model: distilgpt2
Question: what is capital of France?
Answer: France has a population that consists mainly in French citizens. T
he average number for
Confidence: unknown
--------------------------------------------------
Model: roBERTa
Question: What is the capital of France?
Answer: Paris
Confidence: 0.9508629441261292
--------------------------------------------------
Model: roBERTa
Question: What is the capital of England?
Answer: Paris
Confidence: 5.0110968707883785e-09
--------------------------------------------------
```

COMMENTARY ON DIFFERENT WAYS OF ANSWERING QUESTIONS

- Two approaches to answer factual questions are explored:
  - Generative model **DistilGPT2**
  - Task specific extractive model **roBERTa**
- Generative models (DistilGPT2) are trained for **next-token prediction**, not knowledge retrieval
  - They generate plausible text, which may not be correct
- Extractive question answering models like **roBERTa** are fine-tuned on a different paradigm
  - Instead of generating the next token, the model predicts the most likely **span of tokens** within the context that can answer the question
  - These are **encoder-only models (unlike GPT-style decoder models)**
  - With the correct context, this is a significantly more reliable structure for Q&A
- For real-world systems, the need to provide context motivates the use of **Retrieval Augmented Generation (RAG)** systems
  - A knowledge base which forms the context is curated
  - Documents relevant to the question are retrieved and provided as context
  - Answers are generated using the retrieved information
  - This architecture forms the backbone of enterprise copilots and knowledge assistants

Approach	Strength	Limitation
Generative LLM (DistilGPT2)	Can answer without context	May hallucinate
Extractive QA (RoBERTa)	Accurate if answer is in context	Requires user-provided context
RAG systems	Combines retrieval with generation	More complex system design

LLM FOR CREATIVE TASKS

In [22]:
```
# Creative Prompt
prompt_creative = '''
Write a four line poem about artificial intelligence.

Poem:
'''

creative_output = generator(prompt_creative, max_length=80)

processed_text = creative_output[0]["generated_text"]
output = processed_text.replace(prompt_creative, "").strip()
print(output)
```

```
Passing `generation_config` together with generation-related arguments=
({'max_length'}) is deprecated and will be removed in future versions. Ple
ase pass either a `generation_config` object OR all generation parameters
explicitly, but not both.
Setting `pad_token_id` to `eos_token_id`:50256 for open-end generation.
Both `max_new_tokens` (=256) and `max_length`(=80) seem to have been set.
`max_new_tokens` will take precedence. Please refer to the documentation f
or more information. (https://huggingface.co/docs/transformers/main/en/mai
n_classes/text_generation)
"I always wonder if there will ever be a time when I would write a poem th
at I would write in a new context, and that would be the time I would writ
e a poetry that felt very real.
We should see that this, I say, will be the time when I would write a poem
that I would write on top of my computer.
Is there anything that would make this a better place?
I don't know.
I don't know.
Do you really feel that any of us can really have that chance to have some
thing that we can relate to?
I don't know.
I don't know.
Is there anything that would make this a better place?
I don't know.
Do you really feel that any of us can really have that chance to have some
thing that we can relate to?
We should see that this, I say, will be the time when I would write a poem
that I would write on top of my computer.
Is there anything that would make this a better place?
I don't know.
Do you really feel that any of us can really have that chance to have some
thing that we can relate to?
I don't know.
Do you really feel that any
```

In [23]:
```
def write_poem(prompt_creative):
    generation_config = {
            'max_new_tokens': 50,
            'num_return_sequences': 3,
            'temperature': 0.7,
            'top_p': 0.95,
            'do_sample': True,
            'repetition_penalty': 1.3,
            'pad_token_id': 50256
        }
    creative_output = generator(prompt_creative, **generation_config)

    for i, g in enumerate(creative_output):
        text = g["generated_text"].replace(prompt_creative, "").strip()
        print(f"\nOutput {i+1}:")
        print(text)
```

In [24]:
```
write_poem(prompt_creative=prompt_creative)
```

```
Passing `generation_config` together with generation-related arguments=
({'do_sample', 'max_new_tokens', 'pad_token_id', 'repetition_penalty', 'te
mperature', 'num_return_sequences', 'top_p'}) is deprecated and will be re
moved in future versions. Please pass either a `generation_config` object
OR all generation parameters explicitly, but not both.
Both `max_new_tokens` (=50) and `max_length`(=50) seem to have been set. `
max_new_tokens` will take precedence. Please refer to the documentation fo
r more information. (https://huggingface.co/docs/transformers/main/en/main
_classes/text_generation)
Output 1:
I have seen the most interesting thing in this book, and I don't know what
it is that has made me think of myself as an intellectual philosopher or e
ngineer for years now (and still sometimes even after) but then there's no
thing more exciting than

Output 2:
I think I can say that to you, the problem is we are not always looking fo
r answers; it's very hard in our minds and subconscious because of what ha
ppens when one gets bored with something else at some point or another… Wh
at does your brain

Output 3:
"It's very hard to imagine that people would want it if we were not making
the effort of getting us into these kinds, and just doing something else."
This is precisely what you need to know when writing "The Art of Illusio
n," which
```

In [25]:
```
# Creative Prompt
prompt_creative = '''
Write a four line poem about artificial intelligence.

Poem:
Roses are red, violets are blue
The honey's sweet, and so are you
Thou are my love and I am thine
'''

write_poem(prompt_creative=prompt_creative)
```

```
Both `max_new_tokens` (=50) and `max_length`(=50) seem to have been set. `max_new_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)
Output 1:
Let me sing with thee the song of your soul; let us know how far we come from our world! The last time in all mankind was when they were young men who had been raised by their parents on earth before them -- but this is nothing

Output 2:
I will not be afraid to look after your own people! And if this is true then we may have been born from the ashes of our past selves… but at least it was for them that they knew how all life happened—that in every way

Output 3:
And the truth is that we need to look at our faces as if they were real

In this new book The Meaning of Real Science, Michael Risper explains how human beings live their lives in an age when people have no choice but simply choose
```

COMMENTARY ON DIFFERENT WAYS OF GENERATING A POEM

- METHOD 1: Simple Prompt
  - Generated prose instead of poetry
  - Shows repetition
  - Reason:
    - Model is not provided structure
    - Long continuation is allowed / not stopped
- METHOD 2: Controllong Generation Parameter
  - Temperature, top_p, repetation penalty parameter controlled creativity, token generation and stops repetition
  - The outputs were still not poetry, but the generated text is more coherent in each of the outputs
- METHOD 3: Few Shot Prompting
  - An example of expected output was provided
  - Generated more poetic language

NONE OF THE METHODS RELIABLY GENERATED EXACTLY 4 LINES OF POETRY

LESSONS FROM PROMPT ENGINEERING (SUMMARY OF SECTION 5)

- Model objective determines performance
  - Generative model GPT2 does not return good results for specific tasks like answering questions or summarisation
  - Specific models for specific use cases generate better result (RoBERTa for Q&A, Large BART for summarisation)
- Prompt structure influences output

    - In case of creative prompt, providing refined context produced the best results
- Context improves reliability
    - RoBERTa works correctly when context is provided
    - This is the motivation for RAG architecture
- Sampling parameters affect quality
    - Parameters like temperative, repetition penalty, etc. impact coherence of generated text
    - Parameters like temperature can ensure deterministic vs. creative answers

---

Section 6: Load GloVe Word Embeddings

In [26]:
```
glove_txt  = "glove.6B.50d.txt"
glove_zip  = "glove.6B.zip"
glove_url  = "https://nlp.stanford.edu/data/glove.6B.zip"

if not os.path.exists(glove_txt):
    print("Downloading GloVe (≈ 862 MB, one-time)...")

    # Bypass SSL certificate verification (safe for known URLs on trusted
    ssl_ctx = ssl.create_default_context()
    ssl_ctx.check_hostname = False
    ssl_ctx.verify_mode   = ssl.CERT_NONE

    opener = urllib.request.build_opener(
        urllib.request.HTTPSHandler(context=ssl_ctx)
    )
    urllib.request.install_opener(opener)

    urllib.request.urlretrieve(glove_url, glove_zip)

    with zipfile.ZipFile(glove_zip, "r") as z:
        z.extract(glove_txt)
    os.remove(glove_zip)
    print("Done.")
else:
    print("GloVe file already exists. Skipping download.")
```

```
GloVe file already exists. Skipping download.
```

In [27]:
```
print("\nLoading GloVe embeddings (glove-wiki-gigaword-50)...")
model = api.load("glove-wiki-gigaword-50")
print("Embeddings loaded.")
```

```
ERROR:gensim.downloader:caught non-fatal exception while trying to update
gensim-data cache from 'https://raw.githubusercontent.com/RaRe-Technologie
s/gensim-data/master/list.json'; using local cache at '/Users/rajeevkulkar
ni/gensim-data/information.json' instead
Traceback (most recent call last):
  File "/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/
urllib/request.py", line 1344, in do_open
    h.request(req.get_method(), req.selector, req.data, headers,
  File "/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/
http/client.py", line 1336, in request
    self._send_request(method, url, body, headers, encode_chunked)
  File "/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/
http/client.py", line 1382, in _send_request
    self.endheaders(body, encode_chunked=encode_chunked)
  File "/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/
http/client.py", line 1331, in endheaders
    self._send_output(message_body, encode_chunked=encode_chunked)
  File "/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/
http/client.py", line 1091, in _send_output
    self.send(msg)
  File "/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/
http/client.py", line 1035, in send
    self.connect()
  File "/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/
http/client.py", line 1477, in connect
    self.sock = self._context.wrap_socket(self.sock,
                ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/
ssl.py", line 455, in wrap_socket
    return self.sslsocket_class._create(
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/
ssl.py", line 1041, in _create
    self.do_handshake()
  File "/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/
ssl.py", line 1319, in do_handshake
    self._sslobj.do_handshake()
ssl.SSLCertVerificationError: [SSL: CERTIFICATE_VERIFY_FAILED] certificate
verify failed: unable to get local issuer certificate (_ssl.c:1000)

During handling of the above exception, another exception occurred:

Traceback (most recent call last):
  File "/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/
site-packages/gensim/downloader.py", line 199, in _load_info
    info_bytes = urlopen(url).read()
                 ^^^^^^^^^^^^
  File "/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/
urllib/request.py", line 215, in urlopen
    return opener.open(url, data, timeout)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/
urllib/request.py", line 515, in open
    response = self._open(req, data)
               ^^^^^^^^^^^^^^^^^^^^^
  File "/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/
urllib/request.py", line 532, in _open
    result = self._call_chain(self.handle_open, protocol, protocol +
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/
```

```
urllib/request.py", line 492, in _call_chain
    result = func(*args)
             ^^^^^^^^^^^
  File "/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/
urllib/request.py", line 1392, in https_open
    return self.do_open(http.client.HTTPSConnection, req,
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/
urllib/request.py", line 1347, in do_open
    raise URLError(err)
urllib.error.URLError: <urlopen error [SSL: CERTIFICATE_VERIFY_FAILED] cer
tificate verify failed: unable to get local issuer certificate (_ssl.c:100
0)>
```

```
Loading GloVe embeddings (glove-wiki-gigaword-50)...
Embeddings loaded.
```

In [28]:
```
words = ["king", "queen", "diamond"]

for word in words:
    display(Markdown(f"### `{word}`"))
    vector = model[word]
    print("First 10 vector values:")
    vec_df = pd.DataFrame([vector[:10]], columns=[f"d{i}" for i in range(
    display(vec_df.round(4))

    print("Top 5 similar words:")
    similar = model.most_similar(word, topn=5)
    sim_df = pd.DataFrame(similar, columns=["Word", "Similarity Score"])
    sim_df["Similarity Score"] = sim_df["Similarity Score"].round(4)
    display(sim_df)
    print()
```

king

First 10 vector values:

	d0	d1	d2	d3	d4	d5	d6	d7	d8	d9
0	0.5045	0.6861	-0.5952	-0.0228	0.6005	-0.135	-0.0881	0.4738	-0.618	-0.3101

Top 5 similar words:

	Word	Similarity Score
0	prince	0.8236
1	queen	0.7839
2	ii	0.7746
3	emperor	0.7736
4	son	0.7667

queen

First 10 vector values:

	d0	d1	d2	d3	d4	d5	d6	d7	d8	d9
0	0.3785	1.8233	-1.2648	-0.1043	0.3583	0.6003	-0.1754	0.8377	-0.0568	-0.758

Top 5 similar words:

	Word	Similarity Score
0	princess	0.8515
1	lady	0.8051
2	elizabeth	0.7873
3	king	0.7839
4	prince	0.7822

diamond

First 10 vector values:

	d0	d1	d2	d3	d4	d5	d6	d7	d8	d9
0	-0.4958	0.7842	-0.606	1.3967	0.2889	-0.2058	-0.1074	-0.3325	1.3608	0.1509

Top 5 similar words:

	Word	Similarity Score
0	gold	0.7715
1	diamonds	0.7663
2	gem	0.7375
3	silver	0.7210
4	jewel	0.7102

Section 7: Sentence Embeddings

In [29]:
```
sentences = [
    "Artificial intelligence is transforming business.",
    "Machine learning models analyse large datasets.",
    "Deep learning is widely used in image recognition.",
    "Jewellery diamonds are valued for their clarity.",
    "Gold and diamonds are popular luxury items."
]
```

In [30]:
```
def sentence_vector(sentence):
    words = sentence.lower().split()
    valid_words = [word for word in words if word in model]

    if len(valid_words) == 0:
        return np.zeros(50)

    return np.mean([model[word] for word in valid_words], axis=0)
```

In [31]:
```
sentence_vectors = np.array([sentence_vector(s) for s in sentences])
```

Section 8: Sentence Similarity Matrix

In [32]:
```
similarity_matrix = cosine_similarity(sentence_vectors)
```

In [33]:
```
df = pd.DataFrame(similarity_matrix, index=sentences, columns=sentences)
```

In [34]:
```
print("\nSentence Similarity Matrix")
print(df)
```

```
Sentence Similarity Matrix
                                                    Artificial intelligenc
e is transforming business.  \
Artificial intelligence is transforming business.
1.000000
Machine learning models analyse large datasets.
0.788064
Deep learning is widely used in image recognition.
0.912013
Jewellery diamonds are valued for their clarity.
0.658073
Gold and diamonds are popular luxury items.
0.643136

                                                    Machine learning model
s analyse large datasets.  \
Artificial intelligence is transforming business.
0.788064
Machine learning models analyse large datasets.
1.000000
Deep learning is widely used in image recognition.
0.840155
Jewellery diamonds are valued for their clarity.
0.681918
Gold and diamonds are popular luxury items.
0.667673

                                                    Deep learning is widel
y used in image recognition.  \
Artificial intelligence is transforming business.
0.912013
Machine learning models analyse large datasets.
0.840155
Deep learning is widely used in image recognition.
1.000000
Jewellery diamonds are valued for their clarity.
0.758047
Gold and diamonds are popular luxury items.
0.788067

                                                    Jewellery diamonds are
valued for their clarity.  \
Artificial intelligence is transforming business.
0.658073
Machine learning models analyse large datasets.
0.681918
Deep learning is widely used in image recognition.
0.758047
Jewellery diamonds are valued for their clarity.
1.000000
Gold and diamonds are popular luxury items.
0.925351

                                                    Gold and diamonds are
popular luxury items.
Artificial intelligence is transforming business.
0.643136
Machine learning models analyse large datasets.
0.667673
Deep learning is widely used in image recognition.
```

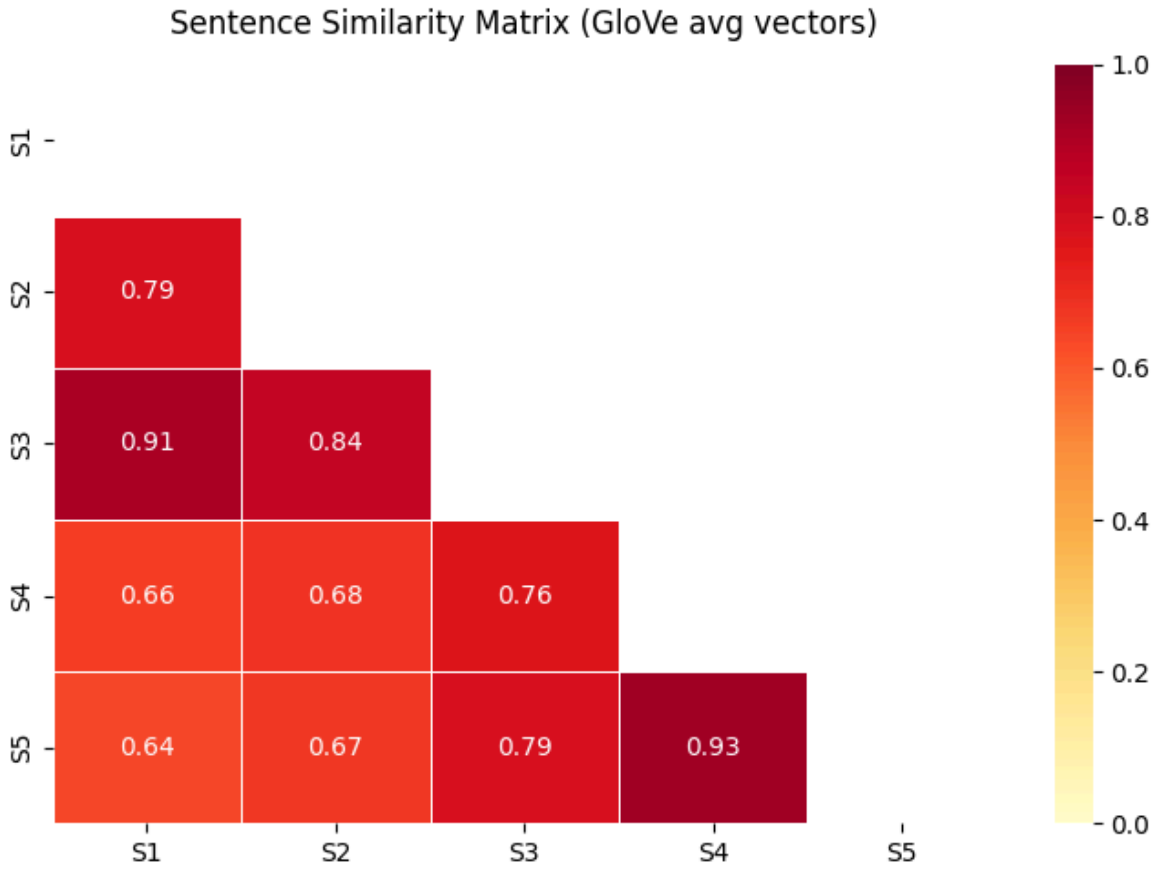
```
0.788067
Jewellery diamonds are valued for their clarity.
0.925351
Gold and diamonds are popular luxury items.
1.000000
```

In [35]:
```
## Short labels for readability
labels = [f"S{i+1}" for i in range(len(sentences))]

sim_df = pd.DataFrame(similarity_matrix, index=labels, columns=labels).ro
mask = np.triu(np.ones_like(sim_df, dtype=bool))

## Heatmap
fig, ax = plt.subplots(figsize=(7, 5))
sns.heatmap(
    sim_df,
    annot=True,
    fmt=".2f",
    cmap="YlOrRd",
    vmin=0,
    vmax=1,
    linewidths=0.5,
    ax=ax,
    mask=mask
)
ax.set_title("Sentence Similarity Matrix (GloVe avg vectors)", pad=12)
plt.tight_layout()
plt.show()

## Legend — which label maps to which sentence
print("\nSentence Legend:")
for label, sentence in zip(labels, sentences):
    print(f"  {label}: {sentence}")
```



```
Sentence Legend:
  S1: Artificial intelligence is transforming business.
  S2: Machine learning models analyse large datasets.
  S3: Deep learning is widely used in image recognition.
  S4: Jewellery diamonds are valued for their clarity.
  S5: Gold and diamonds are popular luxury items.
```

COMMENTARY ON SENTENCE SIMILARITY

- Sample sentences were vectorised using word embedding from Glove
- Cosine similarity between the sentences was computed
- Sentences S1 - S3 are realted to AI/ML and together demostrate high similarity score
- S4-S5 are related to Jewellery and similarly demonstrate high similarity score
- Similarity for all sentences is over 0.6 indicating
  - GloVe embedding are word-level averaged across the sentence
  - Averaging removes word order and context resulting in related sentences appearing similar
  - This is resolved using more modern embeddings like sentence-BERT as BERT architectures capture sematic relationships more explicitly

---

Section 9: Transformer Application Example

In [36]:
```
print("\nSentiment Analysis Example")
sentiment = pipeline("sentiment-analysis")
texts = [
    "This product is excellent and works perfectly.",
    "The service was slow and disappointing."
]
```

```
No model was supplied, defaulted to distilbert/distilbert-base-uncased-fin
etuned-sst-2-english and revision 714eb0f.
Using a pipeline without specifying a model name and revision in productio
n is not recommended.
Sentiment Analysis Example
Loading weights: 100%|██████████| 104/104 [00:00<00:00, 2145.03it/s, Mater
ializing param=pre_classifier.weight]
```

In [37]:
```
results = sentiment(texts)
```

In [38]:
```
for text, result in zip(texts, results):
    print("\nText:", text)
    print("Sentiment:", result)
```

```
Text: This product is excellent and works perfectly.
Sentiment: {'label': 'POSITIVE', 'score': 0.9998756647109985}

Text: The service was slow and disappointing.
Sentiment: {'label': 'NEGATIVE', 'score': 0.9996953010559082}
```

Key Lessons from the Experiments

1. Model architecture determines task performance
    - DistilGPT2 performs poorly for summarisation
    - BART-based models perform significantly better
2. Prompt engineering cannot compensate for model limitations
3. Extractive QA models are reliable when context is provided
4. Retrieval systems (RAG) solve the context problem by supplying relevant information to the model
5. Classical embeddings (GloVe) capture semantic similarity but lack contextual understanding compared to modern transformer embeddings.